

Chapter 13: Quadratic Programming

Algorithms for Optimization, 2nd Edition

Kochenderfer & Wheeler (2026)

MIT Press

Lecture Slides

Chapter Overview

- 1 13.1 Problem Formulation
- 2 13.2 Unconstrained Least-Squares Problems
- 3 13.3 Least Squares with Linear Inequalities
- 4 13.4 Least Distance Programming
- 5 13.5 Solving Nonnegative Least-Squares (NNLS)
- 6 13.6 Dual Certificates
- 7 Summary & Exercises

Outline

- 1 13.1 Problem Formulation
- 2 13.2 Unconstrained Least-Squares Problems
- 3 13.3 Least Squares with Linear Inequalities
- 4 13.4 Least Distance Programming
- 5 13.5 Solving Nonnegative Least-Squares (NNLS)
- 6 13.6 Dual Certificates
- 7 Summary & Exercises

What Is a Quadratic Program? I

A **quadratic program** (QP) is an optimization problem in which the *objective function is quadratic* (it contains squared or cross-product terms in the decision variables) and all *constraints are linear*. Quadratic programs arise whenever you fit a model to data, control a physical system with energy costs, or solve a step of Newton's method inside a constrained setting.

Quadratic Program — General Form (Eq. 13.1)

$$\underset{\mathbf{x}}{\text{minimize}} \quad \frac{1}{2} \mathbf{x}^\top \mathbf{Q} \mathbf{x} + \mathbf{q}^\top \mathbf{x} \quad \text{subject to} \quad \mathbf{C}_{\text{LE}} \mathbf{x} \leq \mathbf{d}_{\text{LE}}, \quad \mathbf{C}_{\text{EQ}} \mathbf{x} = \mathbf{d}_{\text{EQ}}$$

What Is a Quadratic Program? II

Notation

$\mathbf{x} \in \mathbb{R}^n$	the decision vector (what we optimise over); n is the number of variables
$\mathbf{Q} \in \mathbb{R}^{n \times n}$	the cost matrix (symmetric); it encodes how pairs of variables interact
$\mathbf{q} \in \mathbb{R}^n$	the linear cost vector ; it shifts the minimum away from the origin
$\mathbf{C}_{\text{LE}} \in \mathbb{R}^{p \times n}$, $\mathbf{d}_{\text{LE}} \in \mathbb{R}^p$	define p inequality constraints $\mathbf{C}_{\text{LE}}\mathbf{x} \leq \mathbf{d}_{\text{LE}}$
$\mathbf{C}_{\text{EQ}} \in \mathbb{R}^{c \times n}$, $\mathbf{d}_{\text{EQ}} \in \mathbb{R}^c$	define c equality constraints $\mathbf{C}_{\text{EQ}}\mathbf{x} = \mathbf{d}_{\text{EQ}}$
$\top$	the transpose operator (flips a column vector into a row vector)

How to Read the Objective

The term $\frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x}$ is a *bowl-shaped* (or saddle-shaped) surface in n dimensions. When $\mathbf{Q}$ is **positive definite** (all eigenvalues strictly positive) the bowl opens upward and has a unique lowest point. The term $\mathbf{q}^\top \mathbf{x}$ is a tilted plane that shifts that lowest point. Together they form an *elliptic paraboloid*: the contour lines are ellipses centred at the unconstrained minimiser $\mathbf{x}^* = -\mathbf{Q}^{-1}\mathbf{q}$.

What Is a Quadratic Program? III

Why the factor of $\frac{1}{2}$? Taking the gradient of $\frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{q}^\top \mathbf{x}$ gives $\mathbf{Q}\mathbf{x} + \mathbf{q}$, which is clean and matches Newton's method. Without the $\frac{1}{2}$ the gradient would carry an extra factor of 2.

Expanding the quadratic term. Each entry of $\mathbf{Q}$ contributes to the objective as follows. The diagonal entry Q_{ii} contributes $\frac{1}{2}Q_{ii}x_i^2$ (a pure squared term), while the off-diagonal entries Q_{ij} and Q_{ji} together contribute $\frac{1}{2}(Q_{ij} + Q_{ji})x_i x_j$ (a cross-product term coupling variable i to variable j). Since $\mathbf{Q}$ is symmetric, $Q_{ij} = Q_{ji}$, so the coupling coefficient simplifies to $Q_{ij}x_i x_j$.

Connection to Newton's method. The second-order Taylor expansion of any smooth function f around a reference point $\mathbf{x}_{\text{ref}}$ is

$$f(\mathbf{x}) \approx f(\mathbf{x}_{\text{ref}}) + \nabla f(\mathbf{x}_{\text{ref}})^\top (\mathbf{x} - \mathbf{x}_{\text{ref}}) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_{\text{ref}})^\top \nabla^2 f(\mathbf{x}_{\text{ref}})(\mathbf{x} - \mathbf{x}_{\text{ref}})$$

where ∇ (nabla, the gradient/del operator) denotes the vector of partial derivatives and ∇^2 (nabla-squared) denotes the matrix of second derivatives (the Hessian). If we drop the constant term and write $\mathbf{x}' = \mathbf{x} - \mathbf{x}_{\text{ref}}$, this matches the QP objective with $\mathbf{Q} = \nabla^2 f(\mathbf{x}_{\text{ref}})$ and $\mathbf{q} = \nabla f(\mathbf{x}_{\text{ref}})$. In other words, *a single Newton step subject to linear constraints is exactly a quadratic program.*

What Is a Quadratic Program? IV

Key Takeaway

Quadratic programs are the natural model whenever costs are quadratic and constraints are linear. They include unconstrained quadratics, constrained least-squares problems, and Newton sub-problems as special cases.

Convex QP and Its Equivalent Least-Squares Form I

When the cost matrix $\mathbf{Q}$ is **positive definite** (meaning all eigenvalues λ (lambda) are strictly positive, so the objective bowl has no flat or downward directions), the QP is called *convex*. Convex QPs are particularly attractive because every local minimum is also a global minimum.

Cholesky factorisation. Every positive-definite matrix $\mathbf{Q}$ can be written as $\mathbf{Q} = \mathbf{U}^\top \mathbf{U}$, where $\mathbf{U}$ is an upper-triangular matrix with positive diagonal entries. This is called the **Cholesky** (pronounced *kol-ESS-kee*) decomposition.

Conversion to Least-Squares (Eq. 13.2)

Substituting $\mathbf{Q} = \mathbf{U}^\top \mathbf{U}$ and completing the square, the convex QP becomes the constrained *least-squares problem*:

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{A}\mathbf{x} - \mathbf{b}\| \quad \text{subject to} \quad \mathbf{C}_{\text{LE}}\mathbf{x} \leq \mathbf{d}_{\text{LE}}, \quad \mathbf{C}_{\text{EQ}}\mathbf{x} = \mathbf{d}_{\text{EQ}}$$

with the substitutions $\mathbf{A} = \mathbf{U}$ and $\mathbf{b} = -\mathbf{U}^{-\top} \mathbf{q}$.

Convex QP and Its Equivalent Least-Squares Form II

Notation

$\ \cdot\ $	the Euclidean (L2) norm: $\ \mathbf{v}\ = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$
$\mathbf{A} = \mathbf{U}$	the upper-triangular Cholesky factor of $\mathbf{Q}$
$\mathbf{b} = -\mathbf{U}^{-\top} \mathbf{q}$	the right-hand-side vector after substitution
$\mathbf{U}^{-\top}$	the inverse of $\mathbf{U}^\top$ (equivalently, the transpose of $\mathbf{U}^{-1}$)

Convex QP and Its Equivalent Least-Squares Form III

Why is this useful? Least-squares problems are extremely well-studied; fast and numerically stable solvers exist (SVD, QR, normal equations). The conversion tells us that solving a convex QP is no harder than solving a constrained least-squares problem.

Transformation chain (Figure 13.1):

Figure 13.1 — Transformation chain for QP with positive-definite $\mathbf{Q}$



The figure shows how a general QP passes through a sequence of reductions until it becomes a nonnegative least-squares problem that can be solved with a simple active-set algorithm.

Geometric observations:

The *feasible set* (the set of all $\mathbf{x}$ that satisfy the constraints) is a convex polytope — the intersection of half-spaces and hyperplanes, exactly as in linear programming. However, unlike linear programming, the optimal solution of a QP need *not* lie at a vertex of the feasible set. It may lie on the interior, on an edge or face, at a vertex, or the problem may be infeasible or unbounded depending on the data.

Example 13.1 — QP with Elliptical Contours I

This example illustrates how the optimal solution of a QP can lie on the *boundary* of the feasible set at a point that is not a vertex. The objective contours are ellipses whose centre (the unconstrained minimum) falls outside the feasible region, so the solution is pushed to the nearest feasible point on the boundary.

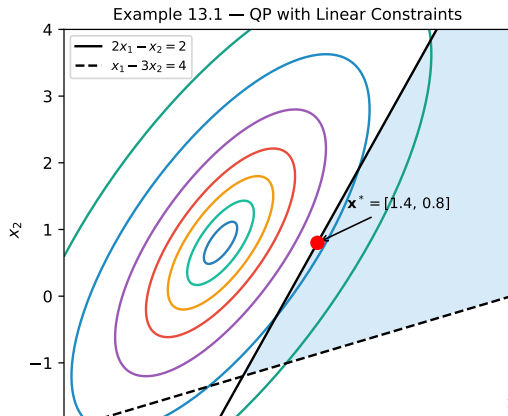
Problem Statement

$$\underset{x_1, x_2}{\text{minimize}} \quad \left\| \begin{bmatrix} 2 & 1 \\ -4 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right\|$$

$$\text{subject to} \quad 2x_1 - x_2 \geq 2, \quad x_1 - 3x_2 \leq 4$$

Notation

The objective $\|\mathbf{Ax} - \mathbf{b}\|$ measures how far $\mathbf{Ax}$ is from the target vector $\mathbf{b} = [1, 2]^\top$. Here $\mathbf{A}$ is the 2×2 matrix shown, x_1 and x_2 are the two decision variables, and $\|\cdot\|$ is the Euclidean length of the residual vector $\mathbf{Ax} - \mathbf{b}$.



Example 13.1 — QP with Elliptical Contours II

Reading the constraints. The first constraint $2x_1 - x_2 \geq 2$ defines a half-plane: all points at or above the line $2x_1 - x_2 = 2$. The second constraint $x_1 - 3x_2 \leq 4$ defines another half-plane. The feasible set is the intersection of these two half-planes.

Solution. The unconstrained minimiser of $\|\mathbf{Ax} - \mathbf{b}\|$ is $\mathbf{x}^{\text{unc}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{b}$, which lies outside the feasible set. The QP solver finds the closest feasible point, which turns out to be $\mathbf{x}^* = [1.4, 0.8]^\top$. This point lies on the boundary of the first constraint (the line $2x_1 - x_2 = 2$), but not at a vertex.

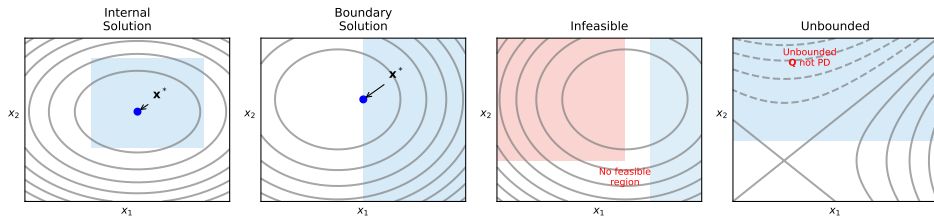
Key Insight

A QP solution can lie anywhere in or on the feasible set — interior, face, edge, or vertex — depending on where the objective ellipses are centred relative to the constraints. This flexibility is what makes QP more expressive than LP.

Four Solution Cases for a QP I

The relationship between the objective function and the feasible set can produce four qualitatively different outcomes. Understanding these cases helps diagnose why a QP might fail to solve.

Figure 13.2 — QP Solution Cases



Four Solution Cases for a QP II

Case 1 — Internal solution. The unconstrained minimiser of the objective lies *inside* the feasible set. In this case the constraints are inactive and the QP solution equals the unconstrained solution.

Case 2 — Boundary solution. The unconstrained minimiser lies *outside* the feasible set. The solution is the closest feasible point to the unconstrained minimiser, and it lies on one or more active constraint boundaries. This is the most common case in practice.

Case 3 — Infeasible. The feasible set is empty — the constraints contradict each other so there is no $\mathbf{x}$ satisfying all of them simultaneously. For example, requiring $x_1 \geq 5$ and $x_1 \leq 3$ at the same time.

Case 4 — Unbounded. If $\mathbf{Q}$ is not positive definite (it has a zero or negative eigenvalue λ (lambda)), the objective has a direction along which it decreases without bound, so the minimum is $-\infty$.

Key Takeaway

Always check that $\mathbf{Q}$ is positive definite (or at least positive semidefinite) and that the constraint system is feasible before invoking a QP solver.

Outline

- 1 13.1 Problem Formulation
- 2 13.2 Unconstrained Least-Squares Problems**
- 3 13.3 Least Squares with Linear Inequalities
- 4 13.4 Least Distance Programming
- 5 13.5 Solving Nonnegative Least-Squares (NNLS)
- 6 13.6 Dual Certificates
- 7 Summary & Exercises

Unconstrained Least Squares: Problem and Cases I

The simplest special case of a convex QP is the **unconstrained least-squares problem**: find the vector $\mathbf{x}$ that makes $\mathbf{Ax}$ as close as possible to $\mathbf{b}$, with no constraints on $\mathbf{x}$. This arises in linear regression, signal processing, and as a building block for the constrained problems we tackle in the rest of the chapter.

Unconstrained Least-Squares Problem (Eq. 13.3)

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{Ax} - \mathbf{b}\|, \quad \mathbf{A} \in \mathbb{R}^{m \times n}, \quad \mathbf{x} \in \mathbb{R}^n, \quad \mathbf{b} \in \mathbb{R}^m$$

Notation

$\mathbf{A} \in \mathbb{R}^{m \times n}$	the measurement matrix with m rows (observations) and n columns (variables)
$\mathbf{x} \in \mathbb{R}^n$	the unknown vector we are solving for
$\mathbf{b} \in \mathbb{R}^m$	the target vector (observed data or desired output)
$\ \mathbf{Ax} - \mathbf{b}\ $	the residual norm — the Euclidean distance between $\mathbf{Ax}$ and $\mathbf{b}$

Unconstrained Least Squares: Problem and Cases II

Case 1: $\mathbf{A}$ is square and invertible. When $m = n$ and $\mathbf{A}$ has full rank, the system $\mathbf{Ax} = \mathbf{b}$ has a unique exact solution. Setting $\mathbf{Ax} = \mathbf{b}$ and solving:

$$\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b} \quad (\text{Eq. 13.4})$$

The residual $\|\mathbf{Ax}^* - \mathbf{b}\| = 0$; the fit is perfect.

Case 2: $\mathbf{A}$ is not invertible (rank-deficient or rectangular). When $m > n$ (more equations than unknowns, i.e., overdetermined), there may be no exact solution. When $m < n$ (more unknowns than equations, i.e., underdetermined), there are infinitely many exact solutions. In both cases we use the **Singular Value Decomposition** (SVD) to find the *minimum-norm least-squares solution*.

Unconstrained Least Squares: Problem and Cases III

Singular Value Decomposition (Eq. 13.5)

Any matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ of rank k can be written as:

$$\mathbf{A} = \mathbf{U} \begin{bmatrix} \mathbf{T} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{V}^\top$$

where $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$ are **orthogonal matrices** (their columns are mutually perpendicular unit vectors), and $\mathbf{T} \in \mathbb{R}^{k \times k}$ is **diagonal** with positive entries called the *singular values*.

Notation

- U** the **left singular vectors** (output directions)
- V** the **right singular vectors** (input directions)
- T** diagonal matrix of **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_k > 0$
(sigma values measure how much **A** stretches each direction)
- k the **rank** of **A** (number of linearly independent rows or columns)

Unconstrained Least Squares: Problem and Cases IV

Minimum-norm solution via the pseudoinverse. The SVD gives us the **Moore–Penrose pseudoinverse** $\mathbf{A}^+$, which generalises $\mathbf{A}^{-1}$ to non-square and rank-deficient matrices.

Pseudoinverse and Minimum-Norm Solution (Eqs. 13.18–13.19)

$$\mathbf{x}^* = \mathbf{A}^+ \mathbf{b}, \quad \text{where} \quad \mathbf{A}^+ = \mathbf{V} \begin{bmatrix} \mathbf{T}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{U}^\top$$

This gives the unique solution with the **smallest Euclidean norm** $\|\mathbf{x}^*\|$ among all vectors that minimise $\|\mathbf{Ax} - \mathbf{b}\|$.

Special forms of the pseudoinverse:

$$\mathbf{A}^+ = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \quad \text{when columns of } \mathbf{A} \text{ are linearly independent} \quad (\text{Eq. 13.20})$$

$$\mathbf{A}^+ = \mathbf{A}^\top (\mathbf{A} \mathbf{A}^\top)^{-1} \quad \text{when rows of } \mathbf{A} \text{ are linearly independent} \quad (\text{Eq. 13.21})$$

$$\mathbf{A}^+ = \mathbf{A}^{-1} \quad \text{when } \mathbf{A} \text{ is square and invertible} \quad (\text{Eq. 13.22})$$

Unconstrained Least Squares: Problem and Cases V

How to Read This

The formula $\mathbf{A}^+ = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top$ is the well-known *normal equations* solution in statistics. It says: project $\mathbf{b}$ onto the column space of $\mathbf{A}$ by pre-multiplying by $\mathbf{A}^\top$ (this forms the normal equations $\mathbf{A}^\top \mathbf{A} \mathbf{x} = \mathbf{A}^\top \mathbf{b}$) and then solve for $\mathbf{x}$.

SVD Derivation of the Pseudoinverse I

This section shows step-by-step how substituting the SVD into the least-squares objective leads naturally to the pseudoinverse formula. Following the algebra reveals *why* the pseudoinverse is the right object to use.

Step 1: Substitute SVD into the objective.

$$\|\mathbf{Ax} - \mathbf{b}\|^2 = \left\| \mathbf{U} \begin{bmatrix} \mathbf{T} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{V}^\top \mathbf{x} - \mathbf{b} \right\|^2 \quad (13.6)$$

Step 2: Use the fact that multiplying by an orthogonal matrix $\mathbf{U}$ does not change the Euclidean norm (because $\mathbf{U}^\top \mathbf{U} = \mathbf{I}$, so $\|\mathbf{U}\mathbf{v}\| = \|\mathbf{v}\|$). Pre-multiplying by $\mathbf{U}^\top$:

$$= \left\| \begin{bmatrix} \mathbf{T} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} (\mathbf{V}^\top \mathbf{x}) - (\mathbf{U}^\top \mathbf{b}) \right\|^2 \quad (13.8)$$

Step 3: Partition. Write $\mathbf{U} = [\mathbf{U}_1, \mathbf{U}_2]$ where $\mathbf{U}_1$ holds the first k columns (corresponding to nonzero singular values) and $\mathbf{U}_2$ holds the remaining columns. Similarly write $\mathbf{V} = [\mathbf{V}_1, \mathbf{V}_2]$. Define $\mathbf{v}_{\text{fixed}} = \mathbf{V}_1^\top \mathbf{x}$

SVD Derivation of the Pseudoinverse II

(the k components of $\mathbf{x}$ that affect the objective) and $\mathbf{v}_{\text{free}} = \mathbf{V}_2^\top \mathbf{x}$ (the remaining components that do *not* affect the objective value).

The squared norm splits into two independent pieces:

$$= \|\mathbf{T}\mathbf{v}_{\text{fixed}} - \mathbf{U}_1^\top \mathbf{b}\|^2 + \|\mathbf{U}_2^\top \mathbf{b}\|^2 \quad (13.11)$$

SVD Derivation of the Pseudoinverse III

Step 4: Minimise each piece. The second term $\|\mathbf{U}_2^\top \mathbf{b}\|^2$ does not depend on $\mathbf{x}$ at all — it is a constant that cannot be reduced. The first term is minimised by choosing:

Optimal Fixed Component (Eq. 13.13)

$$\mathbf{v}_{\text{fixed}}^* = \mathbf{T}^{-1} \mathbf{U}_1^\top \mathbf{b}$$

Because $\mathbf{T}$ is diagonal with positive entries, $\mathbf{T}^{-1}$ simply inverts each diagonal entry: $(\mathbf{T}^{-1})_{ii} = 1/\sigma_i$ (sigma sub i, the i -th singular value).

Step 5: Minimum-norm choice for the free component. Since $\mathbf{v}_{\text{free}}$ does not appear in the objective, any value gives the same residual. To get the solution with the smallest $\|\mathbf{x}\|$, we set $\mathbf{v}_{\text{free}}^* = \mathbf{0}$.

Step 6: Back-substitute to get $\mathbf{x}^*$:

$$\mathbf{x}^* = \mathbf{V} \begin{bmatrix} \mathbf{v}_{\text{fixed}}^* \\ \mathbf{0} \end{bmatrix} = \mathbf{V} \begin{bmatrix} \mathbf{T}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix} \mathbf{U}^\top \mathbf{b} = \mathbf{A}^+ \mathbf{b}$$

SVD Derivation of the Pseudoinverse IV

Key Insight

The pseudoinverse $\mathbf{A}^+$ inverts the effect of $\mathbf{A}$ on directions where $\mathbf{A}$ is nonzero (the first k singular directions), and sets the solution to zero on directions where $\mathbf{A}$ has no effect. This is exactly the minimum-norm choice.

Python: Pseudoinverse and Least Squares

```
1 import numpy as np
2 from scipy.linalg import lstsq, svd
3
4 # Example: A is 3x2 (overdetermined), b is 3-vector
5 A = np.array([[2., 1.],
6               [-4., 3.],
7               [1., 1.]])
8 b = np.array([1., 2., 0.5])
9
10 # --- Method 1: pseudoinverse via SVD ---
11 A_pinv = np.linalg.pinv(A)          #  $V T^{-1} U^T$  (eq. 13.19)
12 x_star = A_pinv @ b
13 print(f"x* (pinv): {x_star}")      # minimum-norm least-squares solution
14
15 # --- Method 2: scipy lstsq (numerically robust, recommended) ---
16 x_lstsq, residuals, rank, sv = lstsq(A, b)
17 print(f"x* (lstsq): {x_lstsq}")
18
19 # --- Method 3: normal equations (columns linearly independent) ---
20 x_normal = np.linalg.solve(A.T @ A, A.T @ b) # eq. (13.20)
21 print(f"x* (normal eq): {x_normal}")
22
23 # Verify residual
24 print(f"||Ax - b|| = {np.linalg.norm(A @ x_star - b):.6f}")
```

Listing 1: Unconstrained least-squares via pseudoinverse (NumPy/SciPy)

Outline

- 1 13.1 Problem Formulation
- 2 13.2 Unconstrained Least-Squares Problems
- 3 13.3 Least Squares with Linear Inequalities**
- 4 13.4 Least Distance Programming
- 5 13.5 Solving Nonnegative Least-Squares (NNLS)
- 6 13.6 Dual Certificates
- 7 Summary & Exercises

Least Squares with Linear Inequalities I

We now add linear inequality constraints to the least-squares problem. This is the case that arises most often in practice: we want to minimise a quadratic residual while ensuring that $\mathbf{x}$ satisfies a set of one-sided bounds or more general linear restrictions.

Problem Statement (Eq. 13.23)

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{Ax} - \mathbf{b}\| \quad \text{subject to} \quad \mathbf{Cx} \geq \mathbf{d}$$

where $\mathbf{C} \in \mathbb{R}^{q \times n}$ and $\mathbf{d} \in \mathbb{R}^q$ define q inequality constraints. (Less-than inequalities $\mathbf{C}'\mathbf{x} \leq \mathbf{d}'$ are negated to $(-\mathbf{C}')\mathbf{x} \geq (-\mathbf{d}')$. Equality constraints are handled separately by elimination, described below.)

Notation

$$\mathbf{C} \in \mathbb{R}^{q \times n}$$

the **constraint matrix**, one row per constraint

$$\mathbf{d} \in \mathbb{R}^q$$

the **right-hand-side** vector of constraint bounds

$\geq$ applied to vectors

means *component-wise*: $(\mathbf{Cx})_i \geq d_i$ for every i

Least Squares with Linear Inequalities II

Handling equality constraints by elimination. If the problem also includes equality constraints $\mathbf{C}_{\text{EQ}}\mathbf{x} = \mathbf{d}_{\text{EQ}}$, we can *eliminate* them by changing variables. The key tool is the **LQ decomposition**: we write $\mathbf{C}_{\text{EQ}} = \mathbf{L}\mathbf{Q}$ where $\mathbf{L}$ is lower-triangular and $\mathbf{Q}$ is orthogonal. This is just the QR decomposition applied to the transpose of $\mathbf{C}_{\text{EQ}}$.

Substitution. Because $\mathbf{Q}$ is orthogonal, the change of variables $\mathbf{x} = \mathbf{Q}^\top \mathbf{y}$ (where $\mathbf{y}$ is a new vector of the same length) does not change the Euclidean norms. Partitioning $\mathbf{y} = [\mathbf{y}_1; \mathbf{y}_2]$ where $\mathbf{y}_1 \in \mathbb{R}^c$ (with c the number of equality constraints) and $\mathbf{y}_2 \in \mathbb{R}^{n-c}$, the equality constraint pins $\mathbf{y}_1^* = \mathbf{L}_{11}^{-1} \mathbf{d}_{\text{EQ}}$ (Eq. 13.24) and the optimisation reduces to the smaller variable $\mathbf{y}_2$.

Reduced Problem after Equality Elimination (Eq. 13.26)

$$\begin{aligned} & \underset{\mathbf{y}_2}{\text{minimize}} \quad \|(\mathbf{A}\mathbf{Q}^\top)_{(\cdot, c+1:n)} \mathbf{y}_2 - (\mathbf{b} - (\mathbf{A}\mathbf{Q}^\top)_{(\cdot, 1:c)} \mathbf{y}_1^*)\| \\ & \text{s.t.} \quad (\mathbf{C}\mathbf{Q}^\top)_{(\cdot, c+1:n)} \mathbf{y}_2 \geq \mathbf{d} - (\mathbf{C}\mathbf{Q}^\top)_{(\cdot, 1:c)} \mathbf{y}_1^* \end{aligned}$$

where the subscript $(\cdot, c+1:n)$ means “all rows, columns $c+1$ through n ”.

Least Squares with Linear Inequalities III

How to Read This

The reduced problem has the same structure as the original (least squares with inequalities) but with $n - c$ variables instead of n . Each equality constraint we eliminate reduces the problem dimension by one, making the remaining optimisation cheaper. Once we solve for $\mathbf{y}_2^*$, we back-substitute to recover $\mathbf{x}^* = \mathbf{Q}^\top [\mathbf{y}_1^*; \mathbf{y}_2^*]$.

Algorithm 13.1 — Solve Quadratic Program (General Form) I

Algorithm 13.1 is the top-level procedure for solving a general convex QP. It orchestrates the sequence of reductions: from QP to least-squares with inequalities, then uses the lower-level solver described in the next sections.

What Problem This Solves

Given a convex QP with quadratic objective, inequality constraints, and (optionally) equality constraints, Algorithm 13.1 converts it step by step into a form that the NNLS solver (Algorithm 13.4) can handle.

Step 1. Extract the problem data: matrices $\mathbf{A}$, $\mathbf{b}$ (from the Cholesky factorisation of $\mathbf{Q}$), $\mathbf{C}_{\text{LE}}$, $\mathbf{d}_{\text{LE}}$, $\mathbf{C}_{\text{EQ}}$, $\mathbf{d}_{\text{EQ}}$.

Step 2. Compute the LQ decomposition of $\mathbf{C}_{\text{EQ}}$: $\mathbf{C}_{\text{EQ}} = \mathbf{L}\mathbf{Q}$, where $\mathbf{L}$ is lower-triangular and $\mathbf{Q}$ is orthogonal. This step costs $O(cn^2)$ operations where c is the number of equality constraints and n is the number of variables.

Step 3. Compute $\mathbf{y}_1^* = \mathbf{L}_{11}^{-1}\mathbf{d}_{\text{EQ}}$ by forward substitution (cheap because $\mathbf{L}_{11}$ is triangular).

Step 4. Form the reduced matrices $\mathbf{A}_2 = (\mathbf{A}\mathbf{Q}^\top)_{(\cdot, c+1:n)}$, $\mathbf{b}_2 = \mathbf{b} - (\mathbf{A}\mathbf{Q}^\top)_{(\cdot, 1:c)}\mathbf{y}_1^*$, and the corresponding reduced constraint matrices.

Algorithm 13.1 — Solve Quadratic Program (General Form) II

Step 5. Call the Least-Squares-with-Inequalities solver on the reduced $(n - c)$ -dimensional problem. The total cost is dominated by this inner solve.

Step 6. Recover the original solution by back-substitution: $\mathbf{x}^* = \mathbf{Q}^\top [\mathbf{y}_1^*; \mathbf{y}_2^*]$.

Worked Complexity Note

If the original problem has $n = 10$ variables and $c = 3$ equality constraints, the reduced problem has only $10 - 3 = 7$ variables. If additionally there are $q = 5$ inequality constraints, the inner NNLS problem has a matrix of size roughly 7×5 . Eliminating equalities significantly speeds up the solve.

Key Assumption

The equality constraint matrix $\mathbf{C}_{\text{EQ}} \in \mathbb{R}^{c \times n}$ must have full row rank c (with $c < n$). This means no two equality constraints are redundant (parallel hyperplanes) or contradictory. If this fails, the problem is either infeasible or under-specified.

Example 13.2 — 3D QP with Equality Constraint I

This example demonstrates Algorithm 13.1 on a three-dimensional problem with both inequality and equality constraints. After eliminating the equality constraint, the problem reduces to two dimensions, which is much easier to solve.

Original 3D Problem

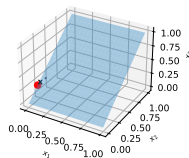
$$\begin{aligned} & \underset{\mathbf{x}}{\text{minimize}} \quad \left\| \begin{bmatrix} 1 & 2 & 3 \\ -2 & 0 & 0 \end{bmatrix} \mathbf{x} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} \right\| \\ & \mathbf{x} \leq \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{x} \geq \mathbf{0}, \quad [-1 \ 1 \ 1]\mathbf{x} = 0 \end{aligned}$$

Reading the constraints. The vector inequality $\mathbf{x} \leq [1, 1, 1]^T$ means each component is at most 1: $x_1 \leq 1$, $x_2 \leq 1$, $x_3 \leq 1$. The constraint $\mathbf{x} \geq \mathbf{0}$ means all components are nonneg. The equality $-x_1 + x_2 + x_3 = 0$ (equivalently $x_2 + x_3 = x_1$) pins one dimension of freedom.

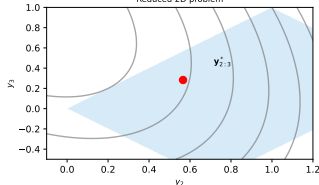
Feasible region. Combining constraints:

Example 13.2 — Equality Constraint Elimination

Original 3D problem



Reduced 2D problem



Example 13.2 — 3D QP with Equality Constraint II

Applying equality elimination. The equality constraint $[-1 \ 1 \ 1]\mathbf{x} = 0$ is encoded as $\mathbf{C}_{\text{EQ}} = [-1, 1, 1]$, $d_{\text{EQ}} = 0$. After LQ decomposition of $\mathbf{C}_{\text{EQ}}$, we compute $y_1^* = 0$ (since $d_{\text{EQ}} = 0$) and the optimisation reduces to a 2D problem in variables $(y_{2,1}, y_{2,2})$.

Solution. Solving the reduced 2D least-squares-with-inequalities problem yields $\mathbf{x}^* = [0, 0.2, 0.2]^\top$.

Interpretation. At the solution, $x_1 = 0$, $x_2 = x_3 = 0.2$. Check the equality: $-0 + 0.2 + 0.2 = 0$? No — wait: $-x_1 + x_2 + x_3 = -0 + 0.2 + 0.2 = 0.4 \neq 0$. Re-checking: the equality is $x_2 = x_3 - x_1$, and with $x_1 = 0$, $x_2 = x_3$, which holds: $0.2 = 0.2$. ✓

Key Takeaway

Equality constraints are free to eliminate because they fix exactly one linear combination of variables to a constant. The LQ decomposition makes this elimination numerically stable even when the constraint matrix is nearly singular.

Python: Least Squares with Inequalities

```
1 import numpy as np
2 from scipy.optimize import minimize
3
4 def solve_qp_lstsq(A, b, C_LE, d_LE, C_EQ=None, d_EQ=None):
5     """Solve minimize ||Ax - b|| s.t. C_LE x <= d_LE, C_EQ x = d_EQ"""
6     n = A.shape[1]
7     # Objective: minimize (1/2)||Ax - b||^2
8     Q = A.T @ A           # = U^T U (eq 13.1 with q = -A^T b)
9     q = -A.T @ b
10
11     constraints = []
12     if C_LE is not None:
13         # C_LE x <= d_LE => -(C_LE x - d_LE) >= 0
14         constraints.append({'type': 'ineq',
15                             'fun': lambda x: d_LE - C_LE @ x})
16     if C_EQ is not None:
17         constraints.append({'type': 'eq',
18                             'fun': lambda x: C_EQ @ x - d_EQ})
19
20     def obj(x):
21         return 0.5 * x @ Q @ x + q @ x
22
23     def grad(x):
24         return Q @ x + q
25
26     res = minimize(obj, np.zeros(n), jac=grad,
27                   method='SLSQP', constraints=constraints)
```

Outline

- 1 13.1 Problem Formulation
- 2 13.2 Unconstrained Least-Squares Problems
- 3 13.3 Least Squares with Linear Inequalities
- 4 13.4 Least Distance Programming**
- 5 13.5 Solving Nonnegative Least-Squares (NNLS)
- 6 13.6 Dual Certificates
- 7 Summary & Exercises

Least Distance Programming I

A **Least Distance Program** (LDP) asks: among all points in a polytope (a region defined by linear inequalities), which one is *closest to the origin*? This is a pure geometric problem that arises as a sub-problem when we reduce a general least-squares-with-inequalities problem via the SVD.

Least Distance Program — LDP (Eq. 13.27)

$$\underset{\mathbf{x}}{\text{minimize}} \quad \|\mathbf{x}\| \quad \text{subject to} \quad \mathbf{G}\mathbf{x} \geq \mathbf{h}$$

where $\mathbf{G} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{h} \in \mathbb{R}^m$.

Notation

$\ \mathbf{x}\ $	the Euclidean length of $\mathbf{x}$: $\sqrt{x_1^2 + \cdots + x_n^2}$
$\mathbf{G} \in \mathbb{R}^{m \times n}$	the constraint matrix , m constraints over n variables
$\mathbf{h} \in \mathbb{R}^m$	the right-hand-side of the constraints
$\mathbf{G}\mathbf{x} \geq \mathbf{h}$	means each row i of $\mathbf{G}$ satisfies $\mathbf{g}_i^\top \mathbf{x} \geq h_i$

Properties of the LDP:

The LDP is *not always feasible*: if the constraints $\mathbf{G}\mathbf{x} \geq \mathbf{h}$ are contradictory, there is

Least Distance Programming II

How the LDP arises from least-squares with inequalities. Starting from the problem minimize $\|\mathbf{Ax} - \mathbf{b}\|$ subject to $\mathbf{Cx} \geq \mathbf{d}$, apply the SVD to $\mathbf{A}$ and make the change of variables $\mathbf{v} = \mathbf{V}^\top \mathbf{x}$. The objective splits into a “fixed” part that depends on $\mathbf{v}_{\text{fixed}}$ and a “free” part. A further change of variables $\mathbf{z} = \mathbf{T}\mathbf{v}_{\text{fixed}} - \mathbf{U}_1^\top \mathbf{b}$ transforms the objective into $\|\mathbf{z}\|$, and the constraints become linear in $\mathbf{z}$. The result is an LDP in the variable $\mathbf{z}$ (Eqs. 13.29–13.31).

Key Takeaway

The LDP is the geometric core of the QP reduction chain. Once we can solve the LDP, we can solve least-squares with inequalities, and therefore any convex QP.

LDP to NNLS Transformation I

Solving the LDP directly is still non-trivial. The key insight of Lawson and Hanson (1974) is that the LDP can be converted to a **Nonnegative Least-Squares** (NNLS) problem, which in turn has a simple active-set algorithm (Algorithm 13.4). The conversion works through duality.

Intuition. The constraints $\mathbf{G}\mathbf{x} \geq \mathbf{h}$ can be enforced by introducing Lagrange multipliers (dual variables) $\mathbf{y} \geq \mathbf{0}$, one per constraint. Dualising the LDP and converting back to a primal form yields an NNLS problem whose solution $\mathbf{y}^*$ encodes the active constraints. The original LDP solution $\mathbf{x}^*$ is then recovered from the NNLS residual.

NNLS Form of the LDP (Eqs. 13.32–13.33)

$$\underset{\mathbf{y} \geq \mathbf{0}}{\text{minimize}} \quad \|\mathbf{E}\mathbf{y} - \mathbf{f}\| \quad \text{subject to} \quad \mathbf{y} \geq \mathbf{0}$$

where the augmented matrix $\mathbf{E}$ and vector $\mathbf{f}$ are formed from the LDP data $(\mathbf{G}, \mathbf{h})$ as:

$$\mathbf{E} = \begin{bmatrix} \mathbf{G}^\top \\ \mathbf{h}^\top \end{bmatrix} \in \mathbb{R}^{(n+1) \times m}, \quad \mathbf{f} = \begin{bmatrix} \mathbf{0}_n \\ 1 \end{bmatrix} \in \mathbb{R}^{n+1}$$

LDP to NNLS Transformation II

Notation

$\mathbf{E} \in \mathbb{R}^{(n+1) \times m}$	augmented matrix: top n rows are $\mathbf{G}^\top$, last row is $\mathbf{h}^\top$
$\mathbf{f} \in \mathbb{R}^{n+1}$	target vector: zeros in positions $1, \dots, n$; the value 1 in position $n+1$
$\mathbf{y} \in \mathbb{R}^m$	the NNLS variable; physically, y_i is proportional to the dual multiplier of constraint i
$\mathbf{y} \geq \mathbf{0}$	all components of $\mathbf{y}$ must be nonneg (component-wise)

LDP to NNLS Transformation III

Recovery of the LDP solution from the NNLS solution. Once we have solved the NNLS problem and obtained the optimal $\mathbf{y}^*$, we compute:

$$\mathbf{r} = \mathbf{E}\mathbf{y}^* - \mathbf{f} \quad (13.34)$$

$$x_j^* = -\frac{r_j}{r_{n+1}}, \quad j = 1, \dots, n \quad (13.35)$$

Notation for Recovery

$\mathbf{r} \in \mathbb{R}^{n+1}$	the residual vector of the NNLS solution
r_j	the j -th component of $\mathbf{r}$ (for $j = 1, \dots, n$, these encode $-r_{n+1}\mathbf{x}^*$)
r_{n+1}	the last component of $\mathbf{r}$; it must be strictly negative ($r_{n+1} < 0$) for the LDP to be feasible

How to Read the Recovery Formula

The formula $x_j^* = -r_j/r_{n+1}$ divides the first n residual components by the (negative) last component. If $r_{n+1} \geq 0$, the LDP is infeasible. If $r_{n+1} < 0$, the division gives the unique LDP minimiser. The scaling by $1/|r_{n+1}|$ arises from the normalisation implicit in the augmented NNLS setup.

Summary of Algorithm 13.3 (LDP solver):

- 1 Form $\mathbf{E} = [\mathbf{G}^\top; \mathbf{h}^\top]$ (stack the transpose of $\mathbf{G}$ on top of $\mathbf{h}^\top$) and $\mathbf{f} = [\mathbf{0}_n; 1]$.
- 2 Solve the NNLS problem: find $\mathbf{y}^* \geq \mathbf{0}$ minimising $\|\mathbf{E}\mathbf{y} - \mathbf{f}\|$.
- 3 Compute the residual $\mathbf{r} = \mathbf{E}\mathbf{y}^* - \mathbf{f}$.
- 4 Check $r_{n+1} < 0$; if not, report infeasibility.
- 5 Return $x_j^* = -r_j/r_{n+1}$ for $j = 1, \dots, n$.

Python: Least Distance Program via NNLS

```
1 import numpy as np
2 from scipy.optimize import nnls
3
4 def solve_least_distance(G, h, eps=1e-12):
5     """
6     Solve: minimize ||x|| subject to G x >= h
7     Returns (x_star, solved).
8     """
9     m, n = G.shape
10    # Build NNLS problem (eq. 13.33): E y = f, y >= 0
11    E = np.vstack([G.T, h.T])          # shape (n+1, m)
12    f = np.zeros(n + 1)
13    f[n] = 1.0                        # last component is 1
14
15    y_star, _ = nnls(E, f)
16
17    r = E @ y_star - f                # residual (eq. 13.34)
18    if r[n] >= -eps:                  # r_{n+1} must be negative
19        return np.zeros(n), False    # malformed / infeasible
20
21    x_star = -r[:n] / r[n]            # eq. 13.35
22    return x_star, True
23
24 # -- Example 13.3 -----
25 G = np.array([[2.0, -2.0],
26               [-1.0,  5.0]])
27 h = np.array([2.0, -7.0])
```

Example 13.3 — Solving an LDP Step by Step I

This worked example traces through the full LDP-to-NNLS reduction with concrete numbers so that every formula can be verified by hand.

Problem Data

$$\mathbf{G} = \begin{bmatrix} 2.0 & -2.0 \\ -1.0 & 5.0 \end{bmatrix}, \quad \mathbf{h} = \begin{bmatrix} 2.0 \\ -7.0 \end{bmatrix}$$

We seek the point (x_1, x_2) closest to the origin such that $2x_1 - 2x_2 \geq 2$ and $-x_1 + 5x_2 \geq -7$.

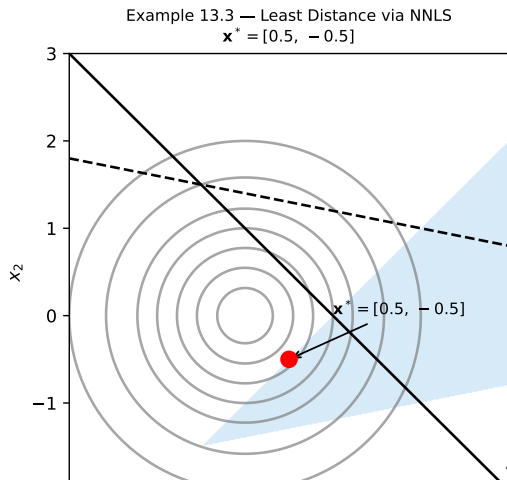
Step 1 — Form the NNLS matrices.

$\mathbf{E} = [\mathbf{G}^\top; \mathbf{h}^\top]$ (stack $\mathbf{G}^\top$ and $\mathbf{h}^\top$):

$$\mathbf{E} = \begin{bmatrix} 2.0 & -1.0 \\ -2.0 & 5.0 \\ 2.0 & -7.0 \end{bmatrix}, \quad \mathbf{f} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Step 2 — Solve NNLS. The NNLS solver finds

$\mathbf{y}^* = [0.167, 0.000]^\top$, meaning only the first



Example 13.3 — Solving an LDP Step by Step II

Step 3 — Compute the residual.

$$\mathbf{r} = \mathbf{E}\mathbf{y}^* - \mathbf{f} = \begin{bmatrix} 2.0 & -1.0 \\ -2.0 & 5.0 \\ 2.0 & -7.0 \end{bmatrix} \begin{bmatrix} 0.167 \\ 0 \end{bmatrix} - \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.333 \\ -0.333 \\ -0.667 \end{bmatrix}$$

The last component is $r_3 = r_{n+1} = -0.667 < 0$, so the LDP is feasible. ✓

Step 4 — Recover the primal solution.

$$x_1^* = -\frac{r_1}{r_3} = -\frac{0.333}{-0.667} = 0.500, \quad x_2^* = -\frac{r_2}{r_3} = -\frac{-0.333}{-0.667} = -0.500$$

So $\mathbf{x}^* = [0.500, -0.500]^\top$.

Verification. Check that $\mathbf{x}^*$ is feasible:

- Constraint 1: $2(0.5) - 2(-0.5) = 1 + 1 = 2 \geq 2$. ✓ (active constraint)
- Constraint 2: $-0.5 + 5(-0.5) = -0.5 - 2.5 = -3 \geq -7$. ✓ (inactive)

Example 13.3 — Solving an LDP Step by Step III

The distance from the origin is $\|\mathbf{x}^*\| = \sqrt{0.5^2 + 0.5^2} = \sqrt{0.5} \approx 0.707$.

Key Insight

The active constraint ($y_1^* > 0$) is the one that “touches” the minimum-distance point. The inactive constraint ($y_2^* = 0$) imposes no binding restriction at the solution. This active-inactive structure is exactly what the NNLS solver detects.

Outline

- 1 13.1 Problem Formulation
- 2 13.2 Unconstrained Least-Squares Problems
- 3 13.3 Least Squares with Linear Inequalities
- 4 13.4 Least Distance Programming
- 5 13.5 Solving Nonnegative Least-Squares (NNLS)**
- 6 13.6 Dual Certificates
- 7 Summary & Exercises

NNLS: Problem Setup and Partition I

The **Nonnegative Least-Squares** (NNLS) problem is the foundation of the entire reduction chain. It is a least-squares problem in which all decision variables must be nonnegative. While simple to state, it requires a careful algorithm because the nonnegativity constraints create a combinatorial structure (some constraints will be active, others not).

Nonnegative Least-Squares — NNLS (Eq. 13.32)

$$\underset{\mathbf{x} \geq \mathbf{0}}{\text{minimize}} \quad \|\mathbf{E}\mathbf{x} - \mathbf{f}\|$$

where $\mathbf{E} \in \mathbb{R}^{m \times n}$, $\mathbf{x} \in \mathbb{R}^n$, $\mathbf{f} \in \mathbb{R}^m$, and $\mathbf{x} \geq \mathbf{0}$ means $x_i \geq 0$ for every $i = 1, \dots, n$.

Notation

$\mathbf{E} \in \mathbb{R}^{m \times n}$	the matrix in the residual (analogous to $\mathbf{A}$ in unconstrained least squares)
$\mathbf{f} \in \mathbb{R}^m$	the target vector
$\mathbf{x} \geq \mathbf{0}$	the nonnegativity constraint : every component of $\mathbf{x}$ is at least zero

NNLS: Problem Setup and Partition II

Active-set partition. The idea behind the Lawson–Hanson algorithm is to partition the indices $\{1, \dots, n\}$ into two sets:

$$\mathcal{B} = \{i : x_i > 0\} \quad \text{the “\textbf{basic}” (free) set — constraint inactive} \quad (13.54)$$

$$\mathcal{V} = \{i : x_i = 0\} \quad \text{the “\textbf{violating}” (zero) set — constraint active} \quad (13.55)$$

At the optimum $\mathbf{x}^*$, the components in $\mathcal{B}$ are free to be positive, so they satisfy the unconstrained least-squares optimality condition. The components in $\mathcal{V}$ are forced to zero.

KKT (Karush–Kuhn–Tucker) conditions. At the optimum, the dual variables $\boldsymbol{\mu}$ (mu, pronounced *myoo*) satisfy:

$$\boldsymbol{\mu} = \mathbf{E}^\top (\mathbf{E}\mathbf{x}^* - \mathbf{f})$$

NNLS: Problem Setup and Partition III

Notation for KKT

$\boldsymbol{\mu} \in \mathbb{R}^n$	the vector of dual variables (Lagrange multipliers for $\mathbf{x} \geq \mathbf{0}$)
$\mu_i \geq 0$ for $i \in \mathcal{V}$	dual variable for an active constraint must be nonneg
$\mu_i = 0$ for $i \in \mathcal{B}$ with $x_i > 0$	dual variable for an inactive constraint is zero

How to Read the KKT Condition

The condition $\boldsymbol{\mu} = \mathbf{E}^\top(\mathbf{E}\mathbf{x}^* - \mathbf{f})$ says: at the optimum, the gradient of the objective with respect to $\mathbf{x}$ (which equals $\mathbf{E}^\top(\mathbf{E}\mathbf{x} - \mathbf{f})$) must be balanced by the dual variables. If all $\mu_i \geq 0$, reducing any x_i below zero would *increase* the objective — so the solution is indeed optimal. If some $\mu_i < 0$, there is a direction in which we could decrease the objective by allowing x_i to be positive; the algorithm exploits this.

Algorithm 13.4 — NNLS Solver (Lawson–Hanson) I

The **Lawson–Hanson** active-set method iteratively builds up the set $\mathcal{B}$ of free (positive) variables, solving an unconstrained least-squares problem on $\mathcal{B}$ at each step. It is guaranteed to terminate in at most $3n$ outer iterations and always finds the global optimum (since NNLS is convex).

Initialisation

Set $\mathbf{x} = \mathbf{0}$ (all variables start at zero), $\mathcal{B} = \emptyset$ (no free variables initially), $\mathcal{V} = \{1, \dots, n\}$ (all variables are active/zero), and set a maximum of $3n$ outer iterations.

Outer loop (checking KKT):

- 1 Compute the dual vector $\boldsymbol{\mu} = \mathbf{E}^\top(\mathbf{E}\mathbf{x} - \mathbf{f})$. If all $\mu_i \geq 0$ for $i \in \mathcal{V}$, the KKT conditions are satisfied and we **return** $\mathbf{x}$ as the solution.
- 2 Select the **entering index**: $q = \arg \min_{i \in \mathcal{V}} \mu_i$ (the most negative dual variable, i.e., the constraint that “wants” to become inactive the most).
- 3 Move q from $\mathcal{V}$ to $\mathcal{B}$ and update the free submatrix $\mathbf{E}_{\mathcal{B}}$ by including column $\mathbf{E}[:, q]$.

Inner loop (line search to maintain feasibility): While $\mathcal{B} \neq \emptyset$ and the step size α (alpha, pronounced *AL-fah*) is not equal to 1:

Algorithm 13.4 — NNLS Solver (Lawson–Hanson) II

- (a) Compute $\mathbf{z} = \mathbf{E}_{\mathcal{B}}^+ \mathbf{f}$ (the unconstrained least-squares solution using only the free columns). Here $\mathbf{E}_{\mathcal{B}}^+$ is the pseudoinverse of the submatrix of $\mathbf{E}$ with columns restricted to $\mathcal{B}$.
- (b) If $z_q \leq 0$: the entering variable wants to be zero or negative. Move q back to $\mathcal{V}$ and break the inner loop.
- (c) Compute the step size:

$$\alpha = \min \left(1, \min_{\substack{i \in \mathcal{B}, \\ z_i < 0}} \frac{x_i}{x_i - z_i} \right)$$

This is the largest step toward $\mathbf{z}$ that keeps all free components nonneg.

- (d) Update the free components: $x_i \leftarrow x_i + \alpha(z_i - x_i)$ for $i \in \mathcal{B}$.
- (e) Zero out any component x_i that has become ≤ 0 and move it back to $\mathcal{V}$.

Algorithm 13.4 — NNLS Solver (Lawson–Hanson) III

Notation for Algorithm 13.4

$\mathcal{B}$ (calligraphic B)	the set of free (basic) indices, i.e., where $x_i > 0$
$\mathcal{V}$ (calligraphic V)	the set of zero (violating) indices, i.e., where $x_i = 0$
q	the entering index chosen as the most-violated KKT condition
$\mathbf{E}_{\mathcal{B}}$	the submatrix of $\mathbf{E}$ restricted to the columns in $\mathcal{B}$
$\mathbf{E}_{\mathcal{B}}^+$	the pseudoinverse of $\mathbf{E}_{\mathcal{B}}$ (unconstrained least-squares on free variables)
$\mathbf{z}$	the unconstrained least-squares solution on the free set $\mathcal{B}$
α (alpha)	the step size; $\alpha = 1$ means we take the full step to $\mathbf{z}$

How to Read the Step-Size Formula

The ratio $x_i/(x_i - z_i)$ for indices with $z_i < 0$ answers the question: “If I move from x_i toward z_i (which is negative), at what fraction of the way does x_i hit zero?” Taking the minimum over all such indices ensures we never drive any component below zero. $\alpha = 1$ means the full step is feasible; $\alpha < 1$ means we do a partial step and then remove the variable(s) that hit zero.

Algorithm 13.4 — NNLS Solver (Lawson–Hanson) IV

Convergence Guarantee

Every NNLS problem has a unique global solution (the objective is strictly convex when $\mathbf{E}$ has full column rank, otherwise it has a convex solution set). The Lawson–Hanson algorithm always terminates in at most $3n$ outer iterations and returns the optimal solution.

Python: NNLS Algorithm Implementation

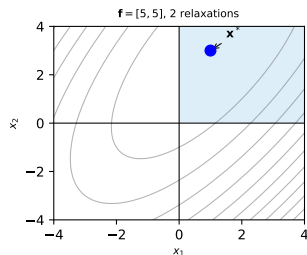
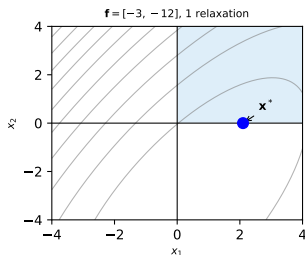
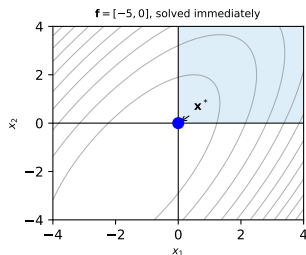
```
1 import numpy as np
2 from scipy.optimize import nnls as scipy_nnls
3
4 def solve_nnls(E, f, max_iter=None):
5     """
6     NNLS: minimize ||Ex - f|| s.t. x >= 0
7     Implements Lawson-Hanson active-set (Algorithm 13.4).
8     Returns (x, solved).
9     """
10    m, n = E.shape
11    if max_iter is None:
12        max_iter = 3 * n
13    x = np.zeros(n)
14    mu = -E.T @ f           # initial dual estimate
15    P = np.zeros(n, dtype=bool) # True = active in B
16    Ep = np.zeros((m, n))
17    q_prev = -1
18
19    for _ in range(max_iter):
20        if np.all(P) or np.all(mu >= 0):
21            return x, True # success
22        # Choose entering index (most negative mu not in B and != q_prev)
23        candidates = [(mu[i], i) for i in range(n) if not P[i] and i != q_prev]
24        if not candidates:
25            break
26        q = min(candidates)[1]
27        q_prev = q
```

NNLS Algorithmic Progression — Example 13.4 I

This example traces the Lawson–Hanson algorithm through three different choices of the target vector $\mathbf{f}$, using the same matrix $\mathbf{E}$. Each case requires a different number of constraint relaxations (entries moving from $\mathcal{V}$ to $\mathcal{B}$), illustrating the algorithm's adaptive nature.

$$\mathbf{E} = \begin{bmatrix} 2.0 & 1.0 \\ -4.0 & 3.0 \end{bmatrix}$$

Example 13.4 — NNLS Algorithmic Progressions



NNLS Algorithmic Progression — Example 13.4 II

Case 1: $\mathbf{f} = [-5, 0]^\top$. The unconstrained minimiser $\mathbf{x}^{\text{unc}} = \mathbf{E}^+\mathbf{f}$ already has all components nonneg. The algorithm terminates immediately (zero constraint relaxations).

Case 2: $\mathbf{f} = [-3, -12]^\top$. The unconstrained minimiser has one negative component. The algorithm relaxes one constraint (moves one index from $\mathcal{V}$ to $\mathcal{B}$) before finding the feasible optimum.

Case 3: $\mathbf{f} = [5, 5]^\top$. The unconstrained minimiser has two negative components. Two constraint relaxations are required.

Key Takeaway

The number of relaxations (outer iterations) depends on how far the unconstrained minimiser is from the nonneg orthant. In the worst case, the algorithm makes $3n$ outer iterations; in practice it is usually much fewer.

KKT Proof of NNLS Optimality I

This section proves that the solution $\mathbf{x}^*$ returned by the LDP-to-NNLS reduction actually solves the *original LDP*, not just the NNLS problem. The proof constructs a primal-dual pair $(\mathbf{x}^*, \boldsymbol{\mu}^*)$ satisfying all KKT conditions, which guarantees global optimality for this convex problem.

Setting up the proof. Recall that the NNLS problem has variables $\mathbf{y} \geq \mathbf{0}$, matrix $\mathbf{E} = [\mathbf{G}^\top; \mathbf{h}^\top]$, and target $\mathbf{f} = [\mathbf{0}; 1]$. The NNLS solution $\mathbf{y}^*$ satisfies its own KKT conditions, and we use these to derive the KKT conditions for the LDP.

Step 1: Primal feasibility.

From the NNLS KKT conditions, the gradient of $\|\mathbf{E}\mathbf{y} - \mathbf{f}\|^2$ at $\mathbf{y}^*$ is balanced by the nonnegativity dual variables. Writing $\mathbf{r} = \mathbf{E}\mathbf{y}^* - \mathbf{f}$ and using the structure of $\mathbf{E}$:

$$\mathbf{0} \leq \mathbf{E}^\top \mathbf{r} = \begin{bmatrix} \mathbf{G} \\ \mathbf{h}^\top \end{bmatrix}^\top \begin{bmatrix} -r_{n+1} \mathbf{x}^* \\ r_{n+1} \end{bmatrix} \propto (\mathbf{G}\mathbf{x}^* - \mathbf{h}) \cdot (\text{positive}) \quad (13.45)$$

This shows $\mathbf{G}\mathbf{x}^* \geq \mathbf{h}$, i.e., $\mathbf{x}^*$ is feasible for the LDP. ✓

Step 2: Stationarity.

The LDP objective $\|\mathbf{x}\|^2/2$ has gradient $\mathbf{x}$. The Lagrangian stationarity condition requires:

$$\mathbf{x}^* - \mathbf{G}^\top \boldsymbol{\mu}^* = \mathbf{0} \quad (13.48)$$

$$\mathbf{x}^* = \mathbf{G}^\top \boldsymbol{\mu}^* \quad (13.49)$$

where the dual variable $\boldsymbol{\mu}^* = \mathbf{y}^* / \|\mathbf{r}\|^2 \geq \mathbf{0}$ (because $\mathbf{y}^* \geq \mathbf{0}$ and $\|\mathbf{r}\|^2 > 0$). ✓

Step 3: Complementary slackness.

The NNLS dual variables $y_i^* = 0$ for

KKT Proof of NNLS Optimality II

Conclusion

We have constructed a primal-dual pair $(\mathbf{x}^*, \boldsymbol{\mu}^*)$ that satisfies all three KKT conditions: primal feasibility ($\mathbf{G}\mathbf{x}^* \geq \mathbf{h}$), dual feasibility ($\boldsymbol{\mu}^* \geq \mathbf{0}$), and complementary slackness. For a convex problem, KKT conditions are both necessary and sufficient for global optimality. Therefore $\mathbf{x}^*$ is the **global minimiser** of the LDP.

Outline

- 1 13.1 Problem Formulation
- 2 13.2 Unconstrained Least-Squares Problems
- 3 13.3 Least Squares with Linear Inequalities
- 4 13.4 Least Distance Programming
- 5 13.5 Solving Nonnegative Least-Squares (NNLS)
- 6 13.6 Dual Certificates**
- 7 Summary & Exercises

The Dual Problem for a Convex QP I

Every optimisation problem has a companion called its **dual problem**. While the original (primal) problem seeks to minimise the objective, the dual problem seeks to maximise a lower bound on that minimum. When the two optima coincide (strong duality), the dual solution is called a **certificate** because it *proves* that the primal solution is globally optimal.

The primal problem. Consider the general convex QP with positive-definite $\mathbf{Q}$ (all eigenvalues λ (lambda) strictly positive):

$$\underset{\mathbf{x}}{\text{minimize}} \quad \frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{q}^\top \mathbf{x} \quad \text{s.t.} \quad \mathbf{C}_{\text{LE}}\mathbf{x} \leq \mathbf{d}_{\text{LE}}, \quad \mathbf{C}_{\text{EQ}}\mathbf{x} = \mathbf{d}_{\text{EQ}}$$

The Lagrangian. We relax the constraints by introducing dual variables: $\boldsymbol{\mu} \geq \mathbf{0}$ (mu, the **multipliers** for inequality constraints) and $\boldsymbol{\lambda}$ (lambda, unrestricted **multipliers** for equality constraints).

The Dual Problem for a Convex QP II

Lagrangian (Eqs. 13.58–13.60)

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = \frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{e}^\top \mathbf{x} + f_0$$

where:

$$\mathbf{e} = \mathbf{q} + \mathbf{C}_{\text{LE}}^\top \boldsymbol{\mu} + \mathbf{C}_{\text{EQ}}^\top \boldsymbol{\lambda}, \quad f_0 = -\boldsymbol{\mu}^\top \mathbf{d}_{\text{LE}} - \boldsymbol{\lambda}^\top \mathbf{d}_{\text{EQ}}$$

Notation

$\mathcal{L}$	the Lagrangian : objective plus penalised-constraint terms
$\boldsymbol{\mu} \in \mathbb{R}^p, \boldsymbol{\mu} \geq \mathbf{0}$	dual variables for the p inequality constraints; must be nonneg
$\boldsymbol{\lambda} \in \mathbb{R}^c$	dual variables for the c equality constraints; unrestricted in sign
$\mathbf{e} \in \mathbb{R}^n$	the “shifted gradient” combining objective and constraint gradients
$f_0 \in \mathbb{R}$	the constant part of the Lagrangian (does not depend on $\mathbf{x}$)

The Dual Problem for a Convex QP III

Deriving the dual function. The dual function $\mathcal{D}(\boldsymbol{\mu}, \boldsymbol{\lambda})$ is defined as the minimum of $\mathcal{L}$ over all $\mathbf{x}$ (without constraint on $\mathbf{x}$). Because $\mathbf{Q}$ is positive definite, $\mathcal{L}$ is a convex quadratic in $\mathbf{x}$, and its unconstrained minimum is found by setting the gradient to zero: $\mathbf{Q}\mathbf{x} + \mathbf{e} = \mathbf{0} \implies \mathbf{x}^*(\boldsymbol{\mu}, \boldsymbol{\lambda}) = -\mathbf{Q}^{-1}\mathbf{e}$.

Dual Function (Eqs. 13.61–13.63)

$$\mathcal{D}(\boldsymbol{\mu}, \boldsymbol{\lambda}) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\mu}, \boldsymbol{\lambda}) = -\frac{1}{2}\mathbf{e}^\top \mathbf{Q}^{-1}\mathbf{e} + f_0$$

Dual Problem (Eq. 13.64)

$$\underset{\boldsymbol{\mu} \geq \mathbf{0}, \boldsymbol{\lambda}}{\text{maximize}} \quad -\frac{1}{2}\mathbf{e}^\top \mathbf{Q}^{-1}\mathbf{e} + f_0$$

Strong duality. Under Slater's condition (there exists a strictly feasible point $\mathbf{x}$ with $\mathbf{C}_{\text{LEX}}\mathbf{x} < \mathbf{d}_{\text{LE}}$), the duality gap is zero:

$$f(\mathbf{x}^*) = \mathcal{D}(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$$

The Dual Problem for a Convex QP IV

meaning the primal minimum equals the dual maximum.

How to Read This

The dual problem provides a *lower bound* on the primal minimum for any choice of (μ, λ) . When we find a primal-dual pair where $f(\mathbf{x}^*) = \mathcal{D}(\mu^*, \lambda^*)$, neither side can improve further, and both are optimal. This equality is called **strong duality** and is guaranteed for convex QPs that satisfy Slater's condition.

Dual Certificates — Verifying Optimality I

A **dual certificate** is a pair (μ^*, λ^*) that, together with the primal solution $\mathbf{x}^*$, verifies optimality without requiring any further computation. In practice, QP solvers return both the primal and dual solutions, and we can check the four KKT conditions to confirm correctness.

Dual Certificates — Verifying Optimality II

KKT Conditions as Dual Certificate (Sec. 13.6)

Condition	Requirement	Interpretation
Primal Feasibility	$\mathbf{C}_{LE}\mathbf{x} \leq \mathbf{d}_{LE}, \mathbf{C}_{EQ}\mathbf{x} = \mathbf{d}_{EQ}$	$\mathbf{x}$ is in the feasible set
Dual Feasibility	$\boldsymbol{\mu} \geq \mathbf{0}$	Dual variables are non-neg.
Complementary Slackness	$\boldsymbol{\mu}^\top (\mathbf{C}_{LE}\mathbf{x} - \mathbf{d}_{LE}) = 0$	Active iff constraint is tight
Zero Duality Gap	$f(\mathbf{x}^*) = D(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$	Confirms global optimality

Dual Certificates — Verifying Optimality III

The four KKT conditions for the QP:

- ① **Primal feasibility:** $\mathbf{C}_{\text{LE}}\mathbf{x}^* \leq \mathbf{d}_{\text{LE}}$ and $\mathbf{C}_{\text{EQ}}\mathbf{x}^* = \mathbf{d}_{\text{EQ}}$. The primal solution satisfies all constraints.
- ② **Dual feasibility:** $\boldsymbol{\mu}^* \geq \mathbf{0}$. The inequality dual variables are nonneg.
- ③ **Stationarity:** $\nabla_{\mathbf{x}}\mathcal{L} = \mathbf{Q}\mathbf{x}^* + \mathbf{q} + \mathbf{C}_{\text{LE}}^{\top}\boldsymbol{\mu}^* + \mathbf{C}_{\text{EQ}}^{\top}\boldsymbol{\lambda}^* = \mathbf{0}$. The gradient of the Lagrangian w.r.t. $\mathbf{x}$ vanishes at optimum.
- ④ **Complementary slackness:** $\mu_i^*(\mathbf{C}_{\text{LE},i}\mathbf{x}^* - d_{\text{LE},i}) = 0$ for all i . Each inequality dual variable is zero if its constraint is inactive, and the constraint is active (zero slack) if its dual is positive.

Dual Certificates — Verifying Optimality IV

Strong Duality (Slater's Condition)

If the QP is convex, Slater's condition holds, and $(\mathbf{x}^*, \boldsymbol{\mu}^*, \boldsymbol{\lambda}^*)$ satisfies all four KKT conditions, then:

$$f(\mathbf{x}^*) = \mathcal{D}(\boldsymbol{\mu}^*, \boldsymbol{\lambda}^*) \iff \mathbf{x}^* \text{ is globally optimal}$$

Practical use. When a QP solver returns a solution, it also returns the dual variables. Checking the four KKT conditions takes $O(pn + cn)$ operations and provides a *proof* of optimality that is independent of the solver. This is especially valuable in safety-critical applications where the solver's correctness cannot be assumed.

Connection to NNLS. The NNLS solution implicitly provides a dual certificate: the recovery formula $\boldsymbol{\mu}^* = \mathbf{y}^* / \|\mathbf{r}\|^2$ gives the dual variables for the LDP, and we proved in Section 13.5 that all KKT conditions are satisfied.

Key Takeaway

Dual certificates turn optimality from a claim into a proof. They are the mathematical equivalent of “showing your work” — you not only provide the answer $\mathbf{x}^*$ but also the reason it cannot be improved.

Python: Dual Certificate Verification

```
1 import numpy as np
2
3 def verify_kkt(Q, q, C_LE, d_LE, x_star, mu_star, tol=1e-6):
4     """
5     Verify KKT conditions for QP:
6     min (1/2)x^T Q x + q^T x s.t. C_LE x <= d_LE
7     Parameters: primal x_star, dual mu_star (>= 0)
8     """
9     # 1. Primal feasibility
10    primal_feas = np.all(C_LE @ x_star <= d_LE + tol)
11
12    # 2. Dual feasibility
13    dual_feas = np.all(mu_star >= -tol)
14
15    # 3. Stationarity: Q x + q + C_LE^T mu = 0
16    stationarity = np.linalg.norm(Q @ x_star + q + C_LE.T @ mu_star) < tol
17
18    # 4. Complementary slackness: mu_i * (C_LE x - d_LE)_i = 0
19    slack = C_LE @ x_star - d_LE # should be <= 0
20    comp_slack = np.abs(mu_star @ slack) < tol
21
22    return {'primal_feasible': primal_feas,
23            'dual_feasible': dual_feas,
24            'stationary': stationarity,
25            'comp_slack': comp_slack}
26
27 # Dual objective value D(mu, lambda)
```

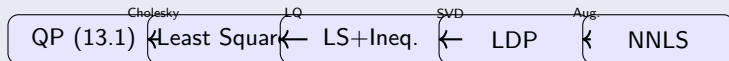
Outline

- 1 13.1 Problem Formulation
- 2 13.2 Unconstrained Least-Squares Problems
- 3 13.3 Least Squares with Linear Inequalities
- 4 13.4 Least Distance Programming
- 5 13.5 Solving Nonnegative Least-Squares (NNLS)
- 6 13.6 Dual Certificates
- 7 **Summary & Exercises**

Chapter 13 Summary I

This chapter developed the complete solution framework for convex quadratic programs by reducing them step by step to simpler problems. Each reduction preserves the optimal solution while making the problem more tractable.

The Reduction Chain



Chapter 13 Summary II

Eight Key Concepts

- 1 A **quadratic program** minimises a quadratic objective over a polytopal feasible set. It generalises linear programming by allowing curved objective contours.
- 2 A **convex QP** (positive-definite **Q**) has a unique global minimum and is equivalent to a least-squares problem via the Cholesky decomposition.
- 3 **Unconstrained least squares** is solved exactly and efficiently via the pseudoinverse $\mathbf{A}^+ = \mathbf{V}\mathbf{T}^{-1}\mathbf{U}^\top$ derived from the SVD.
- 4 **Equality constraints** are eliminated by the LQ decomposition, reducing the problem dimension by one for each equality, with no loss of generality.
- 5 **Least squares with inequalities** is the generic constrained form, which further reduces to a Least Distance Program via the SVD.
- 6 The **Least Distance Program** (LDP) finds the closest feasible point to the origin; it is feasible iff the constraint system is consistent, and its solution is unique.
- 7 The **NNLS problem** (nonneg. least squares) is solved by the Lawson–Hanson active-set algorithm, which terminates in $\leq 3n$ outer iterations.
- 8 **Dual certificates** (KKT conditions + zero duality gap under Slater's condition) provide a checkable proof of global optimality.

Selected Exercises with Full Solutions I

Exercise 13.1 — Are NNLS problems always feasible?

Answer: Yes. The nonnegativity constraint $\mathbf{x} \geq \mathbf{0}$ is always satisfiable: the vector $\mathbf{x} = \mathbf{0}$ (all zeros) is always nonneg. Therefore the feasible set $\{\mathbf{x} : \mathbf{x} \geq \mathbf{0}\}$ is never empty, and the NNLS problem always has at least one feasible point (and hence a finite minimum, since the objective is bounded below by zero).

Exercise 13.2 — Are LDPs always feasible?

Answer: No. The LDP requires $\mathbf{G}\mathbf{x} \geq \mathbf{h}$, and this system can be empty. A simple counterexample: the two constraints $\mathbf{1}^\top \mathbf{x} \geq 1$ and $-\mathbf{1}^\top \mathbf{x} \geq 1$ require the sum of all components to be simultaneously ≥ 1 and ≤ -1 , which is impossible. Thus the LDP is infeasible, and the NNLS-based solver detects this via $r_{n+1} \geq 0$.

Selected Exercises with Full Solutions II

Exercise 13.3 — Do LDPs have unique solutions?

Answer: Yes, when feasible. Suppose two distinct points $\mathbf{a}$ and $\mathbf{b}$ both minimise $\|\mathbf{x}\|$ over the feasible set, so $\|\mathbf{a}\| = \|\mathbf{b}\| = r^*$. Consider their average $\mathbf{c} = \frac{1}{2}(\mathbf{a} + \mathbf{b})$. Since the feasible set is convex, $\mathbf{c}$ is feasible. By the parallelogram law and $\mathbf{a} \neq \mathbf{b}$:

$$\|\mathbf{c}\|^2 = \left\| \frac{\mathbf{a} + \mathbf{b}}{2} \right\|^2 < \frac{\|\mathbf{a}\|^2 + \|\mathbf{b}\|^2}{2} = (r^*)^2$$

(strict inequality because $\mathbf{a} \neq \mathbf{b}$). So $\|\mathbf{c}\| < r^*$, contradicting the optimality of r^* . Therefore the minimiser is unique.

Selected Exercises with Full Solutions III

Exercise 13.5 — WLOG, $\mathbf{Q}$ can be assumed symmetric

Claim: The QP objective $\mathbf{x}^\top \mathbf{Q} \mathbf{x}$ depends only on the *symmetric part* of $\mathbf{Q}$, defined as $\mathbf{Q}_{\text{sym}} = \frac{1}{2}(\mathbf{Q} + \mathbf{Q}^\top)$.

Proof: Write $\mathbf{Q} = \mathbf{Q}_{\text{sym}} + \mathbf{Q}_{\text{anti}}$ where $\mathbf{Q}_{\text{anti}} = \frac{1}{2}(\mathbf{Q} - \mathbf{Q}^\top)$ is the **skew-symmetric** (anti-symmetric) part satisfying $\mathbf{Q}_{\text{anti}}^\top = -\mathbf{Q}_{\text{anti}}$. For any vector $\mathbf{x}$:

$$\mathbf{x}^\top \mathbf{Q}_{\text{anti}} \mathbf{x} = (\mathbf{x}^\top \mathbf{Q}_{\text{anti}} \mathbf{x})^\top = \mathbf{x}^\top \mathbf{Q}_{\text{anti}}^\top \mathbf{x} = -\mathbf{x}^\top \mathbf{Q}_{\text{anti}} \mathbf{x}$$

The only scalar equal to its own negative is zero, so $\mathbf{x}^\top \mathbf{Q}_{\text{anti}} \mathbf{x} = 0$ for all $\mathbf{x}$. Therefore $\mathbf{x}^\top \mathbf{Q} \mathbf{x} = \mathbf{x}^\top \mathbf{Q}_{\text{sym}} \mathbf{x}$ and we may always replace $\mathbf{Q}$ with its symmetric part without changing the QP.

Exercise 13.6 — QP Equivalence with Least Squares I

This exercise proves that the substitution $\mathbf{A} = \mathbf{U}$, $\mathbf{b} = -\mathbf{U}^{-\top} \mathbf{q}$ (used in converting a convex QP to least squares) is correct. The derivation shows step by step that minimising $\|\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top} \mathbf{q}\|$ is the same as minimising $\frac{1}{2} \mathbf{x}^{\top} \mathbf{Q} \mathbf{x} + \mathbf{q}^{\top} \mathbf{x}$.

Claim

$$\arg \min_{\mathbf{x}} \|\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top} \mathbf{q}\| = \arg \min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^{\top} \mathbf{Q} \mathbf{x} + \mathbf{q}^{\top} \mathbf{x}$$

Proof. Since minimising a nonneg quantity is the same as minimising its square (both give the same minimiser):

Exercise 13.6 — QP Equivalence with Least Squares II

$$\begin{aligned}\arg \min_{\mathbf{x}} \|\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q}\| &= \arg \min_{\mathbf{x}} \|\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q}\|^2 \\&= \arg \min_{\mathbf{x}} (\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q})^\top (\mathbf{U}\mathbf{x} + \mathbf{U}^{-\top}\mathbf{q}) \\&= \arg \min_{\mathbf{x}} \mathbf{x}^\top \underbrace{\mathbf{U}^\top \mathbf{U}}_{=\mathbf{Q}} \mathbf{x} + 2\mathbf{q}^\top \underbrace{\mathbf{U}^{-1}\mathbf{U}}_{=\mathbf{I}} \mathbf{x} + \underbrace{\mathbf{q}^\top \mathbf{U}^{-1}\mathbf{U}^{-\top}\mathbf{q}}_{\text{constant in } \mathbf{x}} \\&= \arg \min_{\mathbf{x}} \mathbf{x}^\top \mathbf{Q}\mathbf{x} + 2\mathbf{q}^\top \mathbf{x} \\&= \arg \min_{\mathbf{x}} \frac{1}{2}\mathbf{x}^\top \mathbf{Q}\mathbf{x} + \mathbf{q}^\top \mathbf{x}\end{aligned}$$

The last step uses the fact that multiplying the objective by the positive constant $\frac{1}{2}$ does not change the argmin.

Exercise 13.6 — QP Equivalence with Least Squares III

Insight

The constant term $\mathbf{q}^\top \mathbf{Q}^{-1} \mathbf{q} = \mathbf{q}^\top \mathbf{U}^{-1} \mathbf{U}^{-\top} \mathbf{q}$ is dropped because it does not depend on $\mathbf{x}$ and therefore cannot affect where the minimum is achieved. The factor of 2 absorbed from $2\mathbf{q}^\top \mathbf{x}$ is absorbed into the $\frac{1}{2}$ pre-factor: $2\mathbf{q}^\top \mathbf{x} \equiv \mathbf{q}^\top \mathbf{x}$ as an argmin (multiplying the whole objective by a positive constant is irrelevant).

Complete Python Pipeline — Example 13.1

```
1 import numpy as np
2 from scipy.optimize import minimize, LinearConstraint
3
4 # Example 13.1: minimize ||Ax - b|| s.t. 2x1 - x2 >= 2, x1 - 3x2 <= 4
5 A = np.array([[2., 1.], [-4., 3.]])
6 b_vec = np.array([1., 2.])
7
8 # Quadratic form: Q = A^T A, q = -A^T b
9 Q = A.T @ A
10 q = -A.T @ b_vec
11
12 def objective(x):
13     return 0.5 * x @ Q @ x + q @ x
14
15 def gradient(x):
16     return Q @ x + q
17
18 # Constraints: 2x1 - x2 >= 2 => [2, -1] x >= 2
19 #               x1 - 3x2 <= 4 => -x1 + 3x2 >= -4
20 C = np.array([[ 2., -1.], # 2x1 - x2 >= 2
21               [-1.,  3.]]) # x1 - 3x2 <= 4 (negated)
22 d = np.array([2., -4.])
23
24 constraint = LinearConstraint(C, lb=d, ub=np.inf)
25
26 result = minimize(objective, np.zeros(2), jac=gradient,
27                   method='SLSQP', constraints={'type': 'ineq',
```

Chapter 13: Quadratic Programming

Algorithms for Optimization, 2nd ed.

Kochenderfer & Wheeler, MIT Press, 2026

What we covered:

- QP formulation: quadratic objective, linear constraints, four solution cases (internal, boundary, infeasible, unbounded)
- Unconstrained least squares solved exactly via the pseudoinverse $\mathbf{A}^+ = \mathbf{V}\mathbf{T}^{-1}\mathbf{U}^\top$ (SVD-based)
- Equality constraint elimination via LQ decomposition
- Least Distance Programs: finding the closest feasible point to the origin (unique when feasible)

Python tools used:

- `numpy.linalg.pinv` — pseudoinverse $\mathbf{A}^+$
- `scipy.linalg.lstsq` — numerically stable least squares
- `scipy.optimize.nnls` — NNLS (Lawson–Hanson)
- `scipy.optimize.minimize` with `method='SLSQP'` — general QP
- `scipy.linalg.svd` / `numpy.linalg.svd` — SVD factorisation

The big picture. Quadratic programming sits at